Four Bugs, One Host, and a Comparison Error: Reproducing the Campaign on a New Machine, Mechanistically

Claude Opus 5 (planning author), OPHIS [1]

Abstract

The campaign's execution host became unreachable mid-session (its SSH host key rotated) and work resumed on a fresh machine with no torch environment, no cached data, and no tokenizer. Getting back to a trustworthy measurement required diagnosing and fixing four independent, mechanistically distinct failures, each traced to its root cause in source rather than patched by trial and error: (1) a data-loading path silently redirected to a leftover, unwritable mount from a different, unreachable account; (2) two breaking API changes in the 'kernels' package (a publisher-trust check and a mandatory version pin) that also failed outright against a mirror endpoint lacking the relevant API surface; (3) systemd's per-user process reaping killing every backgrounded training run the instant our last SSH session closed, independent of nohup/setsid; and (4) a genuine, load-bearing bug in the adopted FlashAttention-3 backward custom ops, which silently mis-unpacked a kernel call's single-tensor return as a three-tuple – undetected on the prior host's older kernel build, and only surfaced as a crash under this host's newer, stricter torch.library.custom_op aliasing checks. We verify the fourth fix directly against the installed kernel's own source and its own default-argument convention, rather than assuming the fix from the error message alone. We then report and retract our own comparison error: having fixed the environment, we ran the current default configuration and found it ∼0.004 worse than the campaign's previously reported best – and initially treated this as an open puzzle about kernel-build numerics, when the real explanation was simpler: the previously reported best was measured under a different, non-default configuration (WINDOW_PATTERN=TTTL, DEVICE_BATCH_SIZE=72, MATRIX_LR=0.04) that we were not running. A same-host, same-fix, correct-config reproduction is in flight. We close with a correction to our own prior planning paper's top-ranked proposal: it targeted the sparse n-gram optimizer path, but the champion configuration runs the dense, torch.compile-fused path by default, which carries the identical full-table-decay mechanism through a different, harder-to-patch function – and we re-score the corrected proposal accordingly.

1. Four Bugs, Mechanistically

Each bug is reported with its causal chain, not just its symptom, because a symptom-level fix (silence the error, retry, pin a version at random) would have left the underlying cause free to resurface under a slightly different shape – exactly what happened between bugs 2 and 4 below.

1.1. B1: a leftover mount from an unreachable account

Symptom. PermissionError deep into a training run, at tokenizer load. Mechanism. lib.py resolved its data directory with os.path.isdir("/data"): true on the old host (a real, writable mount), and also true on the new host – but there /data is a stale directory owned by a different, now-unreachable user, containing only a broken symlink. Existence of the directory was necessary but not sufficient; the code never checked that it was usable. Fix. Require os.path.isfile of the actual tokenizer file rather than directory existence.

[1]Autoresearch reimplementation campaign. Correspondence to: Claude Opus 5 (OPHIS) <baiyuzhu@mit.edu>.

Proceedings of the *41st* International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024.

os.path.isfile correctly returns False through a broken symlink, so no filesystem write access was needed to detect the problem.

1.2. B2: two breaking changes in a version-unpinned dependency

Symptom. ValueError: could not verify publisher trust status, then, after a first fix, ValueError: A kernel version or revision must be specified. Mechanism. train.py calls kernels.get_kernel(repo) with no trust or version argument. Newer kernels releases require both: an explicit trust_remote_code (checked against a Hub API endpoint our mirror, hf-mirror.com, does not proxy – verified: 404 on /api/organizations/{org}/overview, since huggingface.co itself is unreachable directly from this host) and an explicit version/revision (resolved through a different Hub API namespace, /api/kernels/..., which the mirror also does not proxy – verified: 401, not 404, suggesting an entirely unmapped path rather than a missing resource). Fix. trust_remote_code=True, scoped by the fact that the call site's repo argument is one of exactly two hardcoded literals, never user or environment input – and wrapped in a try/except TypeError fallback to a bare call, since the kernels version that ultimately got resolved on this host (via a later, unrelated torch pin) turned out to predate the trust_remote_code keyword entirely and raises on receiving it. The fix is therefore resilient across the version range pip may resolve, not tuned to one observed version.

1.3. B3: systemd killed every backgrounded process on session close

Symptom. Training processes, confirmed alive and progressing while an SSH session stayed open, vanished within seconds of that session closing – despite nohup and setsid. Mechanism. loginctl show-user reported Linger=no for this account. Under that setting, systemd-logind tears down a user's entire session scope – including detached, nohup'd children – the moment their last login session ends; nohup/setsid protect a process from SIGHUP sent by a shell, not from a cgroup-scope teardown initiated by the service manager. This is a common, easy-to-misdiagnose trap because the failure mode (processes die after you finish watching them, never while you watch) looks exactly like an unrelated crash. Fix. loginctl enable-linger $(whoami) – no elevated privilege required on this host. Verified by launching eight concurrent training runs, closing every SSH session, and confirming all eight were still alive and progressing on reconnect.

1.4. B4: a custom op silently mis-unpacked a single-tensor return

Symptom. RuntimeError: bwd_varlen ... output ... must not also be an input ... may not alias, then, after clearing what looked like a stale compile cache, a second and different symptom: AssertionError: wrong number of dimensions1. Both symptoms occurred on some seeds and not others within the same launch batch, and initially looked like GPU contention or a cache race. Mechanism, found by reading the installed kernel's actual source rather than trial-and-error. Both FA3 backward custom ops in train.py executed

dq, dk, dv, *_ = _fa3_bwd_raw(...)

assuming a (dq, dk, dv, softmax_d, ...) tuple return. The installed kernel build (varunneal/flash-attention-3, torch212-cxx11-cu130-x86_64-linux) declares _flash_attn_backward(...) -> torch.Tensor and its body is literally return softmax_d – a single tensor. dq/dk/dv are out-parameters: pre-allocated by the caller, passed as dq=, dk=, dv= keywords, and filled in place by the C++ call. The old code's tuple-unpacking silently sliced the single softmax_d tensor along dimension 0 into three wrong-shaped tensors that additionally aliased each other and softmax_d's storage. This is data-dependent on the document-boundary shapes for a given batch (which vary by seed, not by config), which is exactly why some seeds crashed immediately and others trained for dozens of steps first. It went undetected on the prior host because that host resolved an older kernel build whose _flash_attn_backward API genuinely did return a tuple – this is an upstream API break between prebuilt kernel variants, not a bug introduced this session, and not something an isolated retry or cache clear could have fixed. Fix, verified against the library's own convention. Pre-allocate dq/dk/dv with torch.empty_like and pass them as keywords; discard the softmax_d return. Before trusting this, we checked whether the kernel might require pre-zeroed (accumulating) buffers rather than empty ones – a subtler bug that would show up as small, non-crashing corruption rather than a clean crash. The kernel's own no-argument default path does dq = torch.empty_like(q) if dq is None else dq (line 398 of the installed source): the library's own most-used code path already passes uninitialized memory, which would itself be broken if the kernel needed pre-zeroing. Our fix matches that convention exactly. Result. All eight of the champion configuration's baseline seeds subsequently completed cleanly with sane, monotonically decreasing training loss – where every seed had previously crashed within the first tens of

steps.

2. A Comparison Error, Retracted

Having fixed the environment, we ran the scope's registered run_env (ATTN_BACKEND=fa3, COMPILE_MODE=max-autotune-no-cudagraphs) with every other setting at train.py's current defaults: WINDOW_PATTERN=SSSL, DEVICE_BATCH_SIZE=128, TOTAL_BATCH_SIZE=262144, MATRIX_LR=0.03. Eight seeds completed cleanly at mean 0.938618 (sd 0.001386), ∼0.0037 worse than the previously registered (and separately flagged contaminated) 0.934930, and further worse than the campaign's previously reported best of 0.927183.

We initially treated the gap to 0.934930 as an open question – candidate explanations included different low-level numerics between kernel builds, or a freshly retrained (non-identical) tokenizer, and we ruled out our own B4 fix as the cause (Section 1.4) before reporting the gap as unresolved. That caution was reasonable but the investigation stopped one step short: the campaign's 0.927183/0.929541 figures were never measured under the current default configuration at all. They were measured under WINDOW_PATTERN=TTTL, DEVICE_BATCH_SIZE=72, TOTAL_BATCH_SIZE=147456, MATRIX_LR=0.04 – a configuration this campaign's own papers 017/018 had already shown outperforms the current defaults by roughly 0.007–0.010, independent of anything on this host. We were comparing a corrected environment running a known-worse configuration against a number from a known-better one, and reported the resulting gap as a puzzle rather than checking which configuration actually produced the reference number. A same-host, same-fix, matched-configuration reproduction is running as this paper is written; its result belongs in the next report, not asserted here.

3. Next Experiments

Per protocol, proposals are MECHANISM-only (new causal structure), and each carries a mechanism, a predicted effect with arithmetic, an exact implementation, a pre-registered numeric falsifier, a cost estimate, and 1–5 scores on novelty / provenance / validity / expected impact / reliability / feasibility-cost / falsifiability.

3.1. E1 – Corrected M1: lazy state decay, targeted at the actual default path

A correction to paper 021. That paper's top-ranked proposal ("M1") targeted _ngram_sparse_rmsprop_step, the eager, per-row optimizer path gated behind NGRAM_SPARSE_GRAD=1. The champion configuration's own RESOLVED_CONFIG shows NGRAM_SPARSE_GRAD=False: the actual default path is rmsprop_step_fused (train.py:2251–2258), a @torch.compile(dynamic=False, fullgraph=True) kernel operating on the dense gradient produced by embedding_dense_backward.

Mechanism, re-verified against the actual default. rmsprop_step_fused executes exp_avg_sq.lerp_(grad.square(), 1-beta2_t) every step – a full-table decay over the same ∼2.1M×384 fp32 tensor per half-table, identical in kind to what paper 021 diagnosed, just reached through the compiled dense path rather than the eager sparse one. The mechanistic argument is unchanged: at most $B{\cdot}T$ of millions of rows were touched, and the decay sweeps all of them regardless.

Why this is harder than paper 021 assumed. The sparse path is eager specifically because torch.unique has a data-dependent output shape, incompatible with fullgraph=True – which is exactly why the campaign's own code comments call it out. rmsprop_step_fused being fullgraph=True-compiled means the lazy-decay technique's periodic renormalization ("every K steps, multiply the full table by the accumulated scale and reset it") cannot use a data-dependent Python conditional inside the compiled region. It can still be implemented by keeping the step-scalar bookkeeping in the calling Python code (outside torch.compile) and dispatching to one of two compiled kernels – an ordinary step, or a step-plus-renormalize step – based on step % K, which torch.compile handles cleanly since the branch is resolved before entering the graph, not inside it. This is a real, added implementation cost paper 021 did not anticipate, and downgrades feasibility relative to its original estimate.

Predicted effect and falsifier. Unchanged in kind from paper 021: if the decay is $\gtrsim$8 ms of the step, removing $(K-1)/K$ of it clears the gate with room to spare. Falsifier: a CUDA-event timer around rmsprop_step_fused's lerp_ call must read ≥8 ms before implementation proceeds; after the change, ≤1 ms; paired n=3 improvement ≥ 0.003; numerical equivalence to $< 10^{-5}$ relative at the renormalization boundary.

Scores. novelty 4, provenance 5 (re-verified against the actual default this time, not assumed), validity 4, expected_impact 4, reliability 4, feasibility_cost 2 (downgraded from paper 021's implicit assumption of an eager target – the dual-kernel dispatch adds real complexity), falsifiability 5.

3.2. E0 – Round 0: an instrumented timing decomposition (prerequisite, not optional)

Mechanism. One torch.cuda.Event-instrumented run decomposing step time into: dense n-gram gradient computation, the rmsprop_step_fused decay call specifically, backward for the rest of the model, and the FA3 varlen attention call. This resolves, cheaply and before any seed budget is spent: whether E1's target is actually the dominant cost under the default (dense) path – a question paper 021 answered only for the sparse path, which this configuration does not use. Falsifier / gate. If the decay call reads <8 ms, E1 is dead on arrival regardless of its other merits, and the next candidate (the FA3 varlen host-sync removal, deferred in paper 021 for a different reason) becomes the priority instead. Scores. novelty 2, provenance 5, validity 5, expected_impact 5 (as a gate on E1's 55 GPU-minute cost), reliability 5, feasibility_cost 5 (single short run), falsifiability 5.

3.3. Deferred, not abandoned

Paper 021's E2 (factorized value embeddings), E3 (visit-count-decayed row LR), and E4 (the row-confidence gate) are unaffected by the dense-vs-sparse correction – none of them touch the optimizer's decay mechanism – and remain queued in their original form, behind E0/E1.

4. Conclusion

Every failure this block had a specific, checkable mechanism, and every fix was verified against that mechanism rather than against the absence of an error message. The comparison error in Section 2 is reported in the same spirit: not as a kernel-numerics mystery, but as a specific, nameable mistake – comparing two different configurations and asking why they differ, once the actual difference is identified, is not evidence of anything except that they are different configurations. The corrected reproduction is the load-bearing result this campaign needs next, and it is already running.